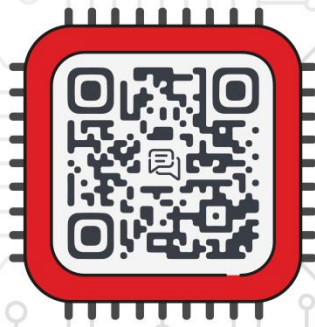




نظري

7



10



1200 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

Spring Boot- Introduction

د. غسان شذود



RB InformatICS; 15/12/2026

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

- تعرفنا في المحاضرة السابقة على إطار العمل spring boot وشرحنا لماذا نحتاجه وبعض الحلول التي يقدمها
- وسنكمل في هذه المحاضرة:

With / Without Spring boot

مثال 1:

1.1

هنا قمنا بإنشاء ServerSocket ولكن سنحتاج الكثير من الأكواد لأجل ال handling socket وتحليل النصوص والتعامل مع الروابط

1.2

بدلاً من فتح المنفذ يدوياً قام spring boot بفتح المنفذ وإدارة الاتصال وإنشاء نظام لإدارة الطلبات المتعددة في صف واحد حيث:

@SpringBootApplication: هنا بدأ البحث عن الإعدادات والمكونات التلقائية

SpringApplication.run(): سطر واحد يقوم بتشغيل كل شيء بما في ذلك الخادم المدمج

@RestController: يحدد أن هذا الكلاس سيتعامل مع طلبات الويب

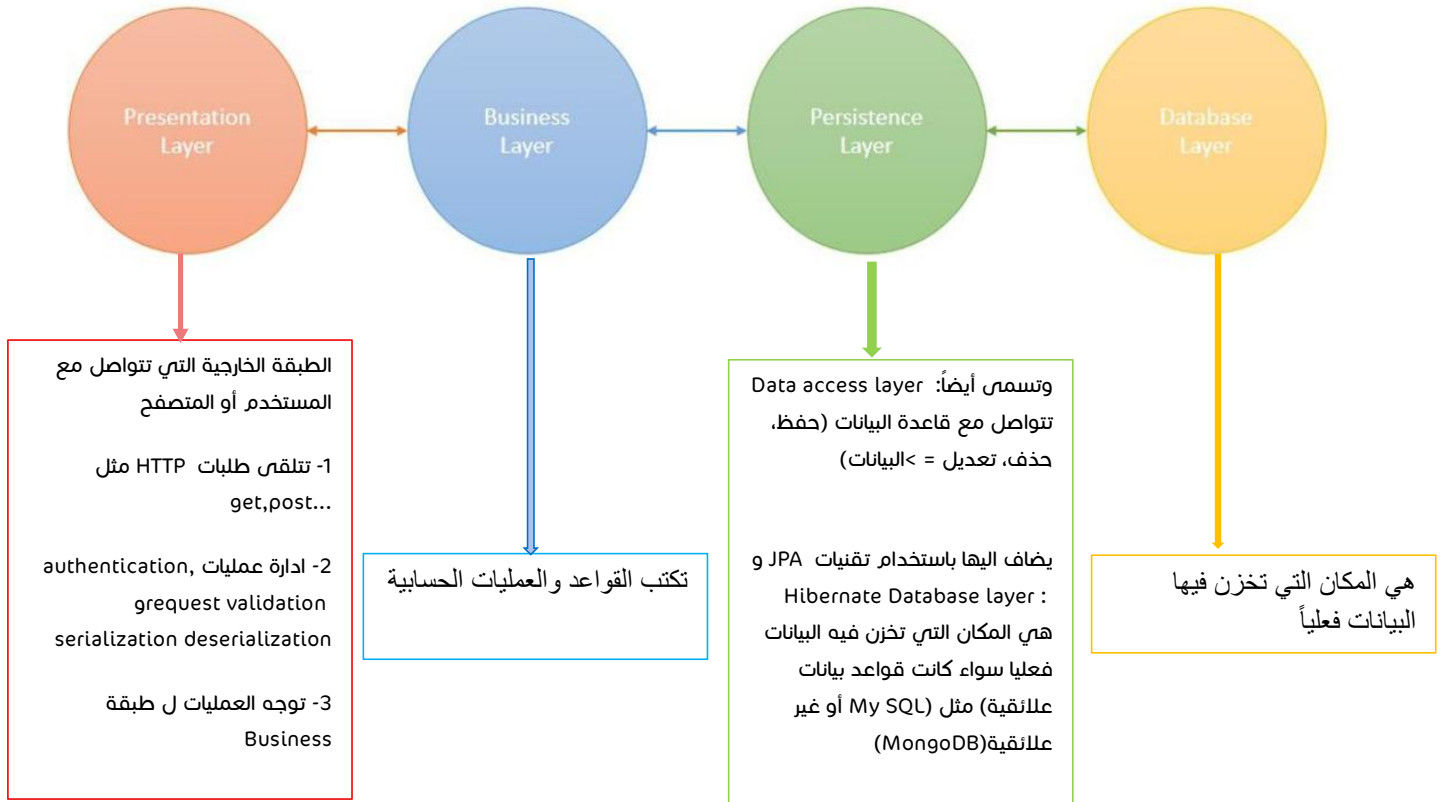
```
// a lot of lines of socket code
ServerSocket server = new ServerSocket(8080);
while (true) {
    Socket client = server.accept();
    // Parse requests, process, respond...
}
```

```
@SpringBootApplication
public class App {
    public static void main (String [] args) {
        SpringApplication.run(App.class, args);
    }
}

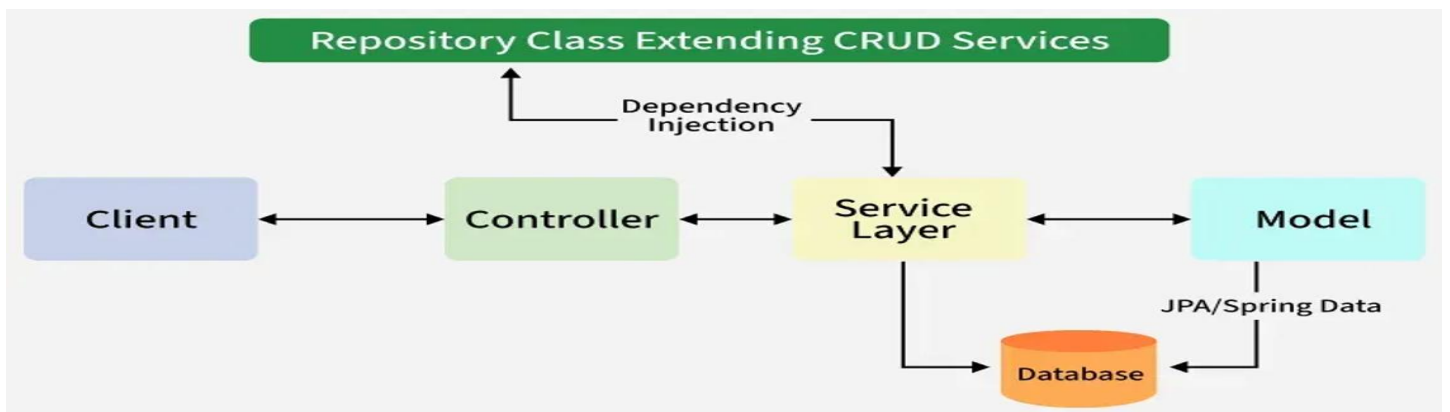
@RestController
class HelloController {
    @GetMapping("/hello")
    public String hello () {
        return "Hello from Spring!";
    }
}
```

Spring Boot- Architecture

يقسم Spring Boot إلى أربع طبقات أساسية منظمة لضمان سهولة التعديل والاختبار :



Spring Boot Flow Architecture



1. يوضح تدفق البيانات (دورة حياة الطلب) داخل النظام

2. العميل Client : يرسل الطلب

3. متحكم Controller : يستقبل الطلب من العميل ويوجهه للوجهة الصحيحة

4. Service Layer : تعالج البيانات وتستخدم هذه الطبقة Dependency injection لجلب الوظائف من طبقة repository

5. Repository : تتواصل مع قاعدة البيانات باستخدام تقنيات من JPA/ SpringData لجلب أو حفظ المعلومات

6. Model : يمثل شكل البيانات التي يتم تبادلها بين الطبقات

7. Data base : مخزن نهائي للمعلومات

The Spring Container

الحاوية وتعرف برمجياً بـ Application Context

ما هو الـ Application Context ؟

هو بيئة تشغيل (Run Time Environment) مسؤولة عن إدارة جميع مكونات التطبيق (Components) يمكن تخيله كـ بيت تعيش فيه جميع الـ objects وتتفاعل مع بعضها البعض يتم إنشاؤه تلقائياً بمجرد استدعاء `SpringApplication.run()` يشبه المصنع الذكي (smart Factory) حيث يقوم بـ:

1. إنشاء الكائنات (beans)
2. الربط بينهما (wiring)
3. إدارة دورة الحياة (مراقبة الكائن من لحظة إنشائه حتى حذفه)
4. توفير الموارد المشتركة

Beans

ما هو؟

هو أي كائن يتم إنشاؤه وتجميعه وإدارته بواسطة Spring Container

الخصائص:

- 1 singleton : يقوم بإنشاء نسخة واحدة من الكائنات فقط
- 2 إدارة دورة الحياة بواسطة Spring
- 3 حقن التبعية Dependencies injected : يقوم بتوصيل الكائنات ببعضها تلقائياً

مثال 2:

```

@Component // Spring will manage this class!"
public class UserService {
    // Spring will create and manage this object
}

```

- عند إضافة @component فوق كلاس وهنا هو User Service فأنت تقول ل Spring تولي إنشاء وإدارة هذا الكلاس

Annotations

هي بيانات وصفية (meta Data) تقوم بإخبار ال spring بكيفية التعامل مع الكود تسمح ببرمجة التعابير (أي كل ما تريد فعله وليس كيف تفعله)

- أهم Annotations :
- @Spring Boot Application : لتعريف كلاس التشغيل الأساسي (Main App Class)
- @Component : لتعريف كائن يقوم Spring بإدارته
- @Service : لمكونات طبقة Business logic
- @Respository : لمكونات Data access
- @Rest Controller : لمكونات طبقة الويب

Dependency Injection (DI)

- ماهي التبعية؟

تعني أن كائن ما (object) يحتاج كائن آخر ليقوم بعمله

مثال يوضح عمل DI :

3.1

- Traditional approach - we create dependencies

```

public class OrderService {
    private PaymentProcessor processor = new PaymentProcessor();
    // Tight coupling!
}

3.2- The Solution - Dependency Injection:
@Service
public class OrderService {
    private final PaymentProcessor processor;
    // Spring provides the dependency
    public OrderService(PaymentProcessor processor) {
        this.processor = processor; // Injected by Spring!
    }
}

```

- لدينا كلاس اسمه order Service ولكي يعمل هذا الكلاس فإنه يحتاج إلى كائن آخر هو payment processor ويتم اعتبار أن payment processor هو تبعية لكلاس order Service
- 3.1 : بالطريقة التقليدية حيث يقوم الكلاس بإنشاء تبعياته بنفسه
- كيف داخل orderService
- نكتب: processor = new paymentProcessor() نهج SpringBoot
- 3.2: هنا تزويد الكلاس بالتبعية من الخارج بواسطة Spring

- فوائد DI:
- ارتباط مرن (loose coupling)
- سهولة الاختبار (testable)
- المرونة (Flexible)
- المسؤولية الواحدة (Single responsibility)

Inversion of Control

- مفهوم يحدد من المسؤول عن إدارة الكود

Traditional Flow (You're in control):

- You → Create objects → Wire them → Manage lifecycle

في التدفق التقليدي:

أي البرمجة التقليدية (بدون استخدام Spring) يكون المطور (أنت) مسؤول عن كل شيء :

- إنشاء الكائنات (create objects)
- توصيل الكائنات (wiring)
- إدارة دورة الحياة (manage life cycle)

Spring Flow (Spring's in control):

- You → Define components → Spring creates → Spring wires → Spring manages

في تدفق Spring :

أي استخدمنا Spring boot في البرمجة

Spring هو المسيطر أي تنتقل السيطرة من المبرمج الى إطار العمل حيث :

أنت تقوم بتحديد المكونات:

مثلا باستخدام @component أو @Service

أما Spring :

Spring هو من ينشئ

Spring هو يقوم بالوصل (wires)

يدير Spring

مبدأ IoC :

▪ framework call you, not you calling framework:

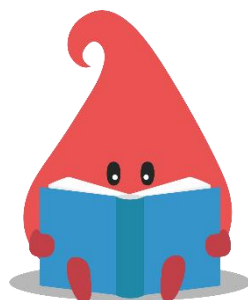
أي بدلاً من أن تقوم أنت باستدعاء دوال إطار العمل يقوم Spring باستدعاء الكود الذي كتبته في الوقت المناسب

▪ you focus on business logic :

تتفرغ كمطور لكتابة القواعد البرمجية والوظائف لمشروع

▪ Spring handles infrastructure :

أي Spring يتولى البنية التحتية (كالمهام المعقدة الروتينية مثل إدارة الذاكرة، فتح اتصالات ...)



مثال 4 :

```
@Service
public class EmailService {
    public void sendEmail(String to, String message) {
        System.out.println("Sending email...");
    }
}

@RestController
public class UserController {
    private final EmailService emailService;

    public UserController(EmailService emailService) {
        this.emailService = emailService;
    }

    @PostMapping("/register")
    public void registerUser() {
        emailService.sendEmail("user@example.com", "Welcome!");
    }

    @PostMapping("/register")
    public void registerUser() {
        // 4. Use the injected dependency
        emailService.sendEmail("user@example.com",
            "Welcome!");
    }
} // END OF UserController
```

- عند قيام المستخدم بالتسجيل يتم استدعاء الدالة register user والتي كانت تستخدم emailService الذي تم حقنه في UserController ليتم إرسال رسالة ترحيب للمستخدم الجديد
- وهنا كلاس EmailService يمثل منطق إرسال إيميلات و UserController يتعامل مع طلبات الويب

ماذا فعل Spring ؟

- (1) إنشاء Beans: إنشاء نسخة لكل من EmailService , UserController
- (2) injection عملية الحقن: قام بتوصيل EmailService داخل UserController
- (3) إدارة دورة الحياة للكائنات

Bean Scopes

Scope	Description	UseCase
Singleton	يتم إنشاء نسخة واحدة فقط من الكائن لكل حاوية Singleton (by default) هو السكوب الافتراضي	الأكثر شيوعاً ويستخدم مع الكلاسات التي لا تخزن بيانات خاصة بالمستخدم مثل Services , Controller
Prototype	يتم إنشاء نسخة جديدة تماماً في كل مرة يطلب فيها التطبيق هذا الكائن من الحاوية	يستخدم مع الكائنات ذات الحالة (Statful objects) مثل عربية التسوق.
Request	يتم إنشاء نسخة واحدة من الكائن لكل طلب HTTP واحد	يستخدم web application حيث يعيش الكائن فقط طوال فترة معالجة الطلب وينتهي بمجرد إرسال الرد للمستخدم
Session	يتم إنشاء نسخة واحدة كل جلسة مستخدم (HTTP Session)	يستخدم لتخزين بيانات جلسة المستخدم (user session data) مثل بيانات تسجيل الدخول أو الأمور التي يجب أن تبقى متاحة طالما المستخدم يتنقل داخل الموقع

*توضيح:

- في prototype ذكرنا مثال عربية التسوق وهنا نقصد أن كل مستخدم يجب أن يحصل على عربية تسوق خاصة فيه .

Three ways to create Beans

- يوفر Spring boot ثلاث طرق أساسية لتعريف الكائنات لكي يتولى إدارتها :

```
@component
Public class MyService{ }
```

1. فحص المكونات Component Scanning : الأكثر شيوعاً حيث يكشف Spring الكلاسات بمجرد وضع @component (أو مشتقاته مثل @service) فوق الكلاس .

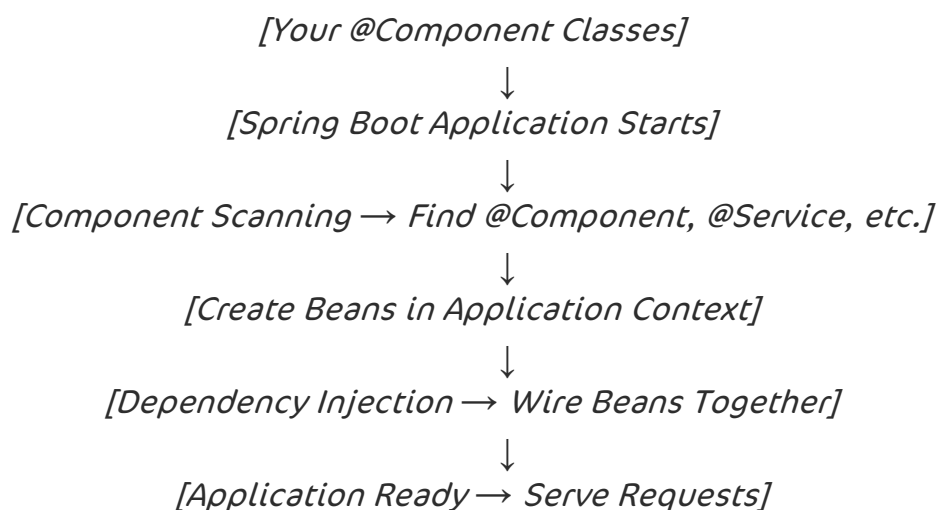
```
Public class Application {
@Bean
Public Data Source
dataSource() {
return new DataSource();}
}
```

2. كلاس الإعداد (Java Configuration class) : يتم إنشاء كلاس خاص يحمل @configuration وتعرف داخله دوال تحمل @Bean لإرجاع نسخ من الكائنات

```
@SpringBootApplication!
Public class MyApp{
@Bean
Public RestTemplate
rest template (){
return new RestTemplate() ;
}
```

3. طريقة المصنع (Factory method) : تعريف دالة تحمل @Bean مباشرة داخل الكلاس الأساسي للتطبيق الذي يحمل Spring Boot Application لإنشاء كائنات محددة .

Complete Picture



- تقوم بتعريف كلاسات تحمل @component
- ينطلق تطبيق SpringBoot
- تبدأ عملية المسح (component Scanning) لإيجاد كافة المكونات
- يتم إنشاء Beans داخل Application context
- يتم حقن التبعية (Dependency injection) لربط ال Beans ببعضها
- يصبح التطبيق جاهز لاستقبال وخدمة الطلبات



The End